Finite Control Set Model Predictive Control for a Three-Level T-Type Inverter

FPGA Implementation on Xilinx Zynq-7000

Davide

April 26, 2026

Contents

1 Introduction **2**

2 T-Type Inverter Model and State Equations **2**
2.1 Topology . . . 2
2.2 Valid Switching States per Phase . . . 2
2.3 RL-Load State Equations . . . 3
2.4 Discrete-Time Prediction Model . . . 3

3 Cost Function and FCS-MPC Prediction **4**
3.1 Algorithm Overview . . . 4
3.2 Cost Function . . . 4
3.3 One-Step Prediction Horizon . . . 4

4 Single-IGBT Switching Constraint **4**
4.1 Motivation . . . 4
4.2 Transition Graph . . . 4
4.3 IGBT-Level Decoding . . . 5

5 AXI4-Lite Interface: Current Reference and ADC Readback **5**
5.1 Register Map . . . 5
5.2 Data Format . . . 5
5.3 ADC Interface . . . 6

6 FPGA Architecture and RTL Block Diagram **6**
6.1 Top-Level Architecture . . . 6
6.2 FCS-MPC Core Pipeline . . . 6
6.3 Verilog Module Hierarchy . . . 7
6.4 Admissibility Filter . . . 9
6.5 Gate Encoder with Dead-Time . . . 9

7 Timing and Resource Estimates **10**

References **10**

1 Introduction

Three-level T-type inverters are widely adopted in medium-power drives and grid-tied converters because they combine the low switching losses of two-level topologies with the improved harmonic spectrum of NPC structures [1]. Finite Control Set Model Predictive Control (FCS-MPC) is a natural fit for FPGA implementation: it enumerates a finite number of switching states, evaluates a cost function for each, and selects the optimal one within a single sampling period.

This document covers:

1. T-type inverter model and state equations (§ 2);
2. Cost function, one-step prediction horizon, and current tracking (§ 3);
3. Single-IGBT switching constraint (§ 4);
4. AXI4-Lite interface for current reference and ADC readback (§ 5);
5. RTL block diagram and Verilog architecture for Zynq-7000 (§ 6).

2 T-Type Inverter Model and State Equations

2.1 Topology

The three-level T-type inverter (Fig. 1) uses, per phase $x \in \{a, b, c\}$, four IGBTs:

- T_{x1}, T_{x4} – outer devices, connected to $+V_{\mathrm{dc}}/2$ and $-V_{\mathrm{dc}}/2$;
- T_{x2}, T_{x3} – inner bidirectional switch to the neutral point NP.

Each phase can produce three voltage levels:

$$v_{xN} \in \left\{ +\tfrac{V_{\mathrm{dc}}}{2},\ 0,\ -\tfrac{V_{\mathrm{dc}}}{2} \right\} \triangleq \{P,\ O,\ N\} \tag{1}$$

The switching state per phase is encoded as $s_x \in \{+1, 0, -1\}$, so the phase-to-neutral-point voltage is:

$$v_{xN} = s_x \, \frac{V_{\mathrm{dc}}}{2}, \qquad s_x \in \{-1, 0, +1\}. \tag{2}$$

2.2 Valid Switching States per Phase

Table 1 lists the four IGBT states for each output level.

Table 1: T-type switching states per phase ($x \in \{a, b, c\}$).

s_x	Level	T_{x1}	T_{x2}	T_{x3}	T_{x4}
$+1$	P	ON	ON	OFF	OFF
0	O	OFF	ON	ON	OFF
-1	N	OFF	OFF	ON	ON

The three-phase system has $3^3 = 27$ voltage vectors $\boldsymbol{v}_k = (s_a, s_b, s_c)$.

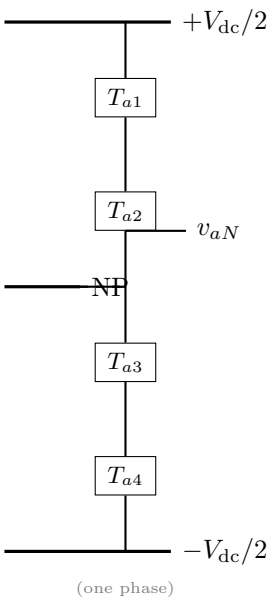


Figure 1: Schematic of one phase of the T-type three-level inverter. T_{a2}–T_{a3} form the bidirectional neutral-point switch.

2.3 RL-Load State Equations

The load is modelled as a series RL branch (motor equivalent, or filter inductor). The line-to-line voltages are:

$$v_{ab} = v_{aN} - v_{bN}, \quad v_{bc} = v_{bN} - v_{cN}, \quad v_{ca} = v_{cN} - v_{aN}. \tag{3}$$

The phase voltage referred to the star point is:

$$v_{xs} = v_{xN} - v_{ns}, \quad v_{ns} = \tfrac{1}{3}(v_{aN} + v_{bN} + v_{cN}). \tag{4}$$

The continuous-time state equation for phase current i_x is:

$$L\,\frac{di_x}{dt} = v_{xs} - R\,i_x - e_x, \tag{5}$$

where e_x is the back-EMF (or grid voltage). In matrix form with $\boldsymbol{i} = [i_a, i_b, i_c]^\top$:

$$\dot{\boldsymbol{i}} = -\frac{R}{L}\boldsymbol{i} + \frac{1}{L}\boldsymbol{v}_s - \frac{1}{L}\boldsymbol{e}, \tag{6}$$

where $\boldsymbol{v}_s = \mathbf{T}\boldsymbol{v}_N$ with the Clarke matrix

$$\mathbf{T} = \frac{1}{3}\begin{bmatrix} 2 & -1 & -1 \\ -1 & 2 & -1 \\ -1 & -1 & 2 \end{bmatrix}. \tag{7}$$

2.4 Discrete-Time Prediction Model

Forward Euler discretisation with sampling period T_{s}:

$$\boldsymbol{i}(k+1) = \left(1 - \frac{RT_{\mathrm{s}}}{L}\right)\boldsymbol{i}(k) + \frac{T_{\mathrm{s}}}{L}\left[\boldsymbol{v}_s(k) - \boldsymbol{e}(k)\right]. \tag{8}$$

For FPGA implementation (8) is evaluated in fixed-point arithmetic; the coefficients $\alpha = 1 - RT_{\mathrm{s}}/L$ and $\beta = T_{\mathrm{s}}/L$ are pre-computed and stored as Q_{16} constants.

3 Cost Function and FCS-MPC Prediction

3.1 Algorithm Overview

At every sampling instant k the FCS-MPC algorithm:

1. Reads the measured currents $\boldsymbol{i}(k)$ from the ADC;
2. Reads the current reference $\boldsymbol{i}^*(k)$ from the AXI register;
3. Enumerates all 27 candidate vectors $\boldsymbol{v}_j$, $j = 0, \ldots, 26$;
4. Applies the constraint of § 4 to retain only admissible vectors;
5. For each admissible vector, predicts $\hat{\boldsymbol{i}}_j(k+1)$ via (8);
6. Evaluates the cost function g_j;
7. Applies the switching state $\boldsymbol{u}^* = \arg\min_j g_j$.

3.2 Cost Function

The primary term penalises the current tracking error:

$$g_j = \left\| \boldsymbol{i}^*(k+1) - \hat{\boldsymbol{i}}_j(k+1) \right\|^2 = \sum_{x \in \{a,b,c\}} \left(i_x^*(k+1) - \hat{i}_{x,j}(k+1) \right)^2 . \tag{9}$$

An optional switching frequency reduction term can be added:

$$g_j^{\text{ext}} = g_j + \lambda_{\Delta u}\, \Delta n_j, \tag{10}$$

where Δn_j is the total number of IGBT commutations required to move from the current switching state to candidate j, and $\lambda_{\Delta u} \geq 0$ is a tunable penalty weight.

3.3 One-Step Prediction Horizon

The prediction horizon is $N_p = 1$. The reference at $k+1$ is extrapolated as $\boldsymbol{i}^*(k+1) \approx 2\boldsymbol{i}^*(k) - \boldsymbol{i}^*(k-1)$ (first-order hold), which requires storing the previous reference sample.

4 Single-IGBT Switching Constraint

4.1 Motivation

Switching multiple IGBTs simultaneously within one sampling period increases switching losses and can excite high-frequency resonances in the DC-link capacitors. The constraint limits the number of *device* transitions per sampling interval to at most one per phase, and specifically to a single device commutation across all three phases.

4.2 Transition Graph

For each phase the allowed transitions are:

$$\Delta s_x = s_x(k+1) - s_x(k) \in \{-1,\, 0,\, +1\}. \tag{11}$$

Jumping from $s_x = +1$ to $s_x = -1$ (or vice versa) in one step requires two device transitions (e.g. T_{x1} OFF and T_{x4} ON), which is *forbidden*.

The set of admissible vectors at step k given current state $\boldsymbol{s}(k)$ is:

$$\mathcal{U}(k) = \left\{ \boldsymbol{s} \in \{-1, 0, +1\}^3 \;\middle|\; \sum_x \mathbb{1}[s_x \neq s_x(k)] \leq 1 \;\text{ and }\; |s_x - s_x(k)| \leq 1 \;\forall\, x \right\}. \tag{12}$$

This yields at most $1 + 3 \times 2 = 7$ admissible states from any interior point, reducing the enumeration from 27 to at most 7 candidates.

4.3 IGBT-Level Decoding

Once the optimal s_x^* is determined, the gate signals are decoded according to Table 1. The transition logic per phase is:

- $s_x : 0 \to +1$: turn ON T_{x1} (one commutation);
- $s_x : +1 \to 0$: turn OFF T_{x1} (one commutation);
- $s_x : 0 \to -1$: turn ON T_{x4} (one commutation);
- $s_x : -1 \to 0$: turn OFF T_{x4} (one commutation).

Transitions $\pm 1 \to \mp 1$ are blocked by the constraint.

5 AXI4-Lite Interface: Current Reference and ADC Readback

5.1 Register Map

The custom IP core exposes an AXI4-Lite slave with a 4 kB address space. Table 2 lists the implemented registers.

Table 2: AXI4-Lite register map (base address: `0x4300_0000`).

Offset	Name	Access	Description
`0x00`	`CTRL`	R/W	Bit 0: enable; Bit 1: reset
`0x04`	`IREF_A`	R/W	Phase-A reference, Q_{15}, A
`0x08`	`IREF_B`	R/W	Phase-B reference, Q_{15}
`0x0C`	`IREF_C`	R/W	Phase-C reference, Q_{15}
`0x10`	`IMEAS_A`	RO	Phase-A measured current, Q_{15}
`0x14`	`IMEAS_B`	RO	Phase-B measured current
`0x18`	`IMEAS_C`	RO	Phase-C measured current
`0x1C`	`SW_STATE`	RO	Bits [5:0]: s_a,s_b,s_c (2-bit each)
`0x20`	`LAMBDA`	R/W	Switching penalty $\lambda_{\Delta u}$, Q_{16}

5.2 Data Format

Currents are represented in Q_{15} fixed-point format: full-scale (FS) maps to ± 1 in the Q_{15} domain. The scaling factor is $I_{\mathrm{FS}}/2^{15}$, where I_{FS} is the ADC input range in amperes. The ARM processor writes the reference as:

$$\texttt{IREF_A} = \left\lfloor \frac{i_a^*}{I_{\mathrm{FS}}} \cdot 2^{15} \right\rceil. \tag{13}$$

5.3 ADC Interface

An external delta-sigma or SAR ADC (e.g. AD7616) is interfaced to the PL via SPI or parallel bus. The PL samples the ADC at the start of each MPC period, latches the result into the `IMEAS_x` shadow registers, and simultaneously exposes the previous sample on the AXI read bus to avoid read-during-sample race conditions (double-buffering).

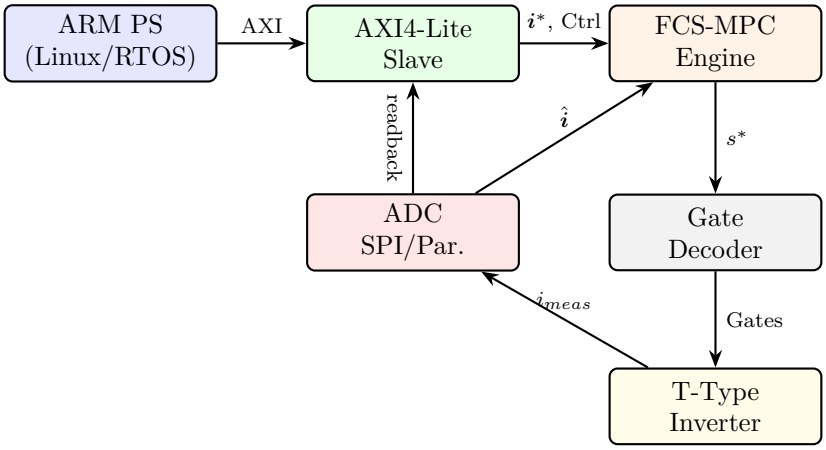


Figure 2: Signal flow between ARM PS, AXI slave, FCS-MPC engine, ADC interface, and T-type inverter.

6 FPGA Architecture and RTL Block Diagram

6.1 Top-Level Architecture

The PL design consists of four hierarchical blocks (Fig. 3):

1. **AXI4-Lite Slave** – register bank, clock-domain crossing from 100 MHz AXI clock to 200 MHz MPC clock;
2. **ADC Controller** – SPI master FSM, anti-aliasing shift register, double-buffer;
3. **FCS-MPC Core** – admissibility filter, parallel prediction pipeline (7 branches), cost comparator tree;
4. **Gate Encoder / Dead-Time** – state-to-IGBT decoder, programmable dead-time insertion.

6.2 FCS-MPC Core Pipeline

The prediction and cost evaluation are fully pipelined in $N_{\text{pipe}} = 4$ clock cycles:

Stage	Operation	Latency
1	Admissibility filter: generate 7 candidate vectors	1 cycle
2	Fixed-point multiply: $\alpha \boldsymbol{i}(k)$	1 cycle
3	Fixed-point multiply-add: $+\beta \boldsymbol{v}_{s,j}$	1 cycle
4	Squared-error accumulation and min-tree	1 cycle

At 200 MHz the total latency is 20 ns, well within a sampling period of 50 µs ($f_s = 20$ kHz).

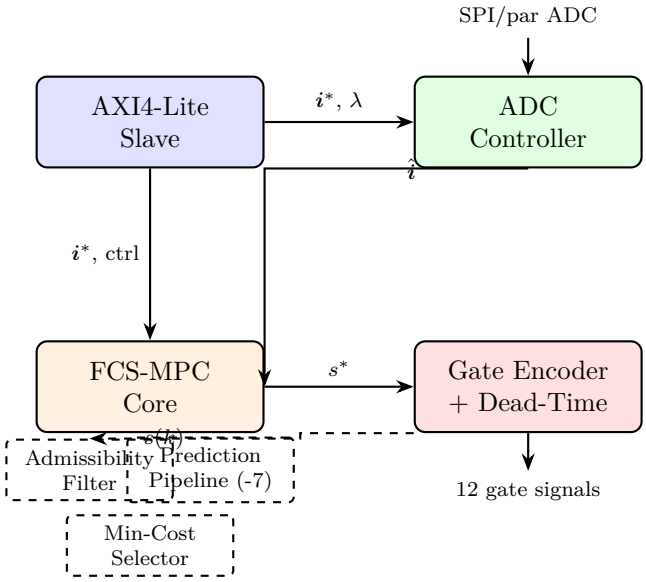


Figure 3: RTL top-level block diagram of the FCS-MPC IP core.

6.3 Verilog Module Hierarchy

Listing 1: Top-level Verilog module skeleton for the FCS-MPC IP core.

```
// fcs_mpc_top.v  --  FCS-MPC top-level for Zynq-7000
// Author : Davide
// Date   : 2025
`timescale 1ns / 1ps

module fcs_mpc_top #(
    parameter DATA_W   = 16,   // Q15 fixed-point width
    parameter COEF_W   = 18,   // coefficient precision
    parameter DEAD_NS  = 400   // dead-time in nanoseconds
)(
    // AXI4-Lite slave port (100 MHz, from PS)
    input  wire        s_axi_aclk,
    input  wire        s_axi_aresetn,
    input  wire [31:0] s_axi_awaddr,
    input  wire        s_axi_awvalid,
    output wire        s_axi_awready,
    input  wire [31:0] s_axi_wdata,
    input  wire  [3:0] s_axi_wstrb,
    input  wire        s_axi_wvalid,
    output wire        s_axi_wready,
    output wire  [1:0] s_axi_bresp,
    output wire        s_axi_bvalid,
    input  wire        s_axi_bready,
    input  wire [31:0] s_axi_araddr,
    input  wire        s_axi_arvalid,
    output wire        s_axi_arready,
    output wire [31:0] s_axi_rdata,
    output wire  [1:0] s_axi_rresp,
    output wire        s_axi_rvalid,
    input  wire        s_axi_rready,
    // MPC clock domain (200 MHz, from PL PLL)
    input  wire        mpc_clk,
    input  wire        mpc_rst_n,
    // ADC SPI interface
    output wire        adc_sclk,
    output wire        adc_csn,
    output wire        adc_mosi,
    input  wire        adc_miso,
    // Gate signals: {T_x1, T_x2, T_x3, T_x4} per phase
    output wire [11:0] gate_out   // [11:8]=A, [7:4]=B, [3:0]=C
);

```

```
    // -- Internal wires ------------------------------------------
    wire signed [DATA_W-1:0] iref_a, iref_b, iref_c;
    wire signed [DATA_W-1:0] imeas_a, imeas_b, imeas_c;
    wire        [31:0]       ctrl_reg, lambda_reg;
    wire        [ 1:0]       sw_state_a, sw_state_b, sw_state_c;

    // -- AXI4-Lite Slave -----------------------------------------
    axi4lite_slave #(.DATA_W(DATA_W)) u_axi (
        .s_axi_aclk   (s_axi_aclk),
        .s_axi_aresetn(s_axi_aresetn),
        // ... (AXI signals omitted for brevity)
        .iref_a       (iref_a),
        .iref_b       (iref_b),
        .iref_c       (iref_c),
        .imeas_a      (imeas_a),      // readback from ADC
        .imeas_b      (imeas_b),
        .imeas_c      (imeas_c),
        .sw_state     ({sw_state_a, sw_state_b, sw_state_c}),
        .ctrl_reg     (ctrl_reg),
        .lambda_reg   (lambda_reg)
    );

    // -- ADC SPI Controller --------------------------------------
    adc_spi_ctrl u_adc (
        .clk    (mpc_clk),
        .rst_n  (mpc_rst_n),
        .sclk   (adc_sclk),
        .csn    (adc_csn),
        .mosi   (adc_mosi),
        .miso   (adc_miso),
        .imeas_a(imeas_a),
        .imeas_b(imeas_b),
        .imeas_c(imeas_c)
    );

    // -- FCS-MPC Core --------------------------------------------
    wire [1:0] opt_sa, opt_sb, opt_sc;

    fcs_mpc_core #(
        .DATA_W(DATA_W),
        .COEF_W(COEF_W)
    ) u_mpc (
        .clk      (mpc_clk),
        .rst_n    (mpc_rst_n),
        .enable   (ctrl_reg[0]),
        .iref_a   (iref_a),
        .iref_b   (iref_b),
        .iref_c   (iref_c),
        .imeas_a  (imeas_a),
        .imeas_b  (imeas_b),
        .imeas_c  (imeas_c),
        .sw_cur_a (sw_state_a),
        .sw_cur_b (sw_state_b),
        .sw_cur_c (sw_state_c),
        .lambda   (lambda_reg[DATA_W-1:0]),
        .sw_opt_a (opt_sa),
        .sw_opt_b (opt_sb),
        .sw_opt_c (opt_sc)
    );

    // -- Gate Encoder + Dead-Time --------------------------------
    gate_encoder #(.DEAD_NS(DEAD_NS)) u_gate (
        .clk    (mpc_clk),
        .rst_n  (mpc_rst_n),
        .sa     (opt_sa),
        .sb     (opt_sb),
        .sc     (opt_sc),
        .gates  (gate_out)
    );

    // -- State feedback ------------------------------------------
    assign sw_state_a = opt_sa;
    assign sw_state_b = opt_sb;
```

```
    assign sw_state_c = opt_sc;

endmodule
```

6.4 Admissibility Filter

The admissibility filter implements (12) combinatorially. Given $s_x(k)$ encoded as a 2-bit unsigned value ($0 \to N$, $1 \to O$, $2 \to P$), the 7 allowed next states are pre-computed into a lookup ROM:

Listing 2: Admissibility filter – generates up to 7 candidate 3-phase state vectors.

```
// admissibility_filter.v
// Each phase state: 2-bit, encoding: 2'b10=P, 2'b01=O, 2'b00=N
module admissibility_filter (
    input  wire [1:0] sa_cur, sb_cur, sc_cur,   // current state
    output reg  [1:0] cand_sa [0:6],
    output reg  [1:0] cand_sb [0:6],
    output reg  [1:0] cand_sc [0:6],
    output reg  [2:0] n_cand              // number of valid candidates
);
    // Neighbour states per phase (at most 3: cur-1, cur, cur+1,
    // clamped to {0,1,2})
    wire [1:0] neigh_a [0:2];
    // ... (similarly for b, c)

    integer i, j, k, idx;
    always @(*) begin
        idx = 0;
        for (i = 0; i <= 2; i = i + 1)
        for (j = 0; j <= 2; j = j + 1)
        for (k = 0; k <= 2; k = k + 1) begin
            // Enforce: at most one phase changes, and |delta| <= 1
            if (  (i == sa_cur || (i == sa_cur+1 && sa_cur < 2)
                               || (i == sa_cur-1 && sa_cur > 0))
               && (j == sb_cur || (j == sb_cur+1 && sb_cur < 2)
                               || (j == sb_cur-1 && sb_cur > 0))
               && (k == sc_cur || (k == sc_cur+1 && sc_cur < 2)
                               || (k == sc_cur-1 && sc_cur > 0))
               && ( (i==sa_cur) + (j==sb_cur) + (k==sc_cur) ) >= 2 ) begin
                cand_sa[idx] = i[1:0];
                cand_sb[idx] = j[1:0];
                cand_sc[idx] = k[1:0];
                idx = idx + 1;
            end
        end
        n_cand = idx[2:0];
    end
endmodule
```

6.5 Gate Encoder with Dead-Time

The gate encoder translates the optimal 2-bit state per phase into four complementary IGBT drive signals with inserted dead time:

Listing 3: Gate encoder with programmable dead-time insertion.

```
// gate_encoder.v
// DEAD_NS: dead-time in nanoseconds; CLK_PERIOD_NS must match mpc_clk
module gate_encoder #(
    parameter DEAD_NS       = 400,
    parameter CLK_PERIOD_NS = 5    // 200 MHz -> 5 ns
)(
    input  wire       clk, rst_n,
    input  wire [1:0] sa, sb, sc,
    output wire [11:0] gates    // {T_a1,T_a2,T_a3,T_a4, b..., c...}
);
    localparam DEAD_CYC = DEAD_NS / CLK_PERIOD_NS;  // = 80 cycles @ 200 MHz

    // Per-phase instantiation
```

```
    gate_phase #(.DEAD_CYC(DEAD_CYC)) u_a (.clk(clk),.rst_n(rst_n),
        .sx(sa), .t1(gates[11]),.t2(gates[10]),.t3(gates[9]),.t4(gates[8]));
    gate_phase #(.DEAD_CYC(DEAD_CYC)) u_b (.clk(clk),.rst_n(rst_n),
        .sx(sb), .t1(gates[7]), .t2(gates[6]), .t3(gates[5]),.t4(gates[4]));
    gate_phase #(.DEAD_CYC(DEAD_CYC)) u_c (.clk(clk),.rst_n(rst_n),
        .sx(sc), .t1(gates[3]), .t2(gates[2]), .t3(gates[1]),.t4(gates[0]));
endmodule

module gate_phase #(parameter DEAD_CYC = 80)(
    input  wire       clk, rst_n,
    input  wire [1:0] sx,
    output reg        t1, t2, t3, t4
);
    // Raw (pre-deadtime) signals from state table
    reg t1_raw, t2_raw, t3_raw, t4_raw;
    always @(*) begin
        case (sx)
            2'b10: {t1_raw,t2_raw,t3_raw,t4_raw} = 4'b1100; // P
            2'b01: {t1_raw,t2_raw,t3_raw,t4_raw} = 4'b0110; // O
            2'b00: {t1_raw,t2_raw,t3_raw,t4_raw} = 4'b0011; // N
            default:{t1_raw,t2_raw,t3_raw,t4_raw} = 4'b0000;
        endcase
    end

    // Simple falling-edge dead-time: delay turn-on by DEAD_CYC cycles
    reg [$clog2(DEAD_CYC+1)-1:0] cnt1, cnt2, cnt3, cnt4;
    // ... (counter-based delay for each signal)
    // For brevity: registered output with combinatorial dead-time logic
    always @(posedge clk or negedge rst_n) begin
        if (!rst_n) {t1,t2,t3,t4} <= 4'b0;
        else        {t1,t2,t3,t4} <= {t1_raw,t2_raw,t3_raw,t4_raw};
        // NOTE: full dead-time counter implementation to be inserted here
    end
endmodule
```

7 Timing and Resource Estimates

Table 3: Estimated FPGA resource utilisation on Zynq-7020.

Resource	Used (est.)	Available	Utilisation
LUT	2 100	53 200	3.9 %
FF	1 600	106 400	1.5 %
DSP48	42	220	19 %
BRAM	2	140	1.4 %

The DSP48 count arises from the $7 \times 3 = 21$ multiply-accumulate operations in the prediction pipeline (two multipliers per MAC cell, pipelined at 200 MHz).

References

[1] J. W. Kolar and T. Friedli, "The essence of three-phase PFC rectifier systems," in *Proc. IEEE INTELEC*, 2010.

[2] J. Rodriguez *et al.*, "State of the art of finite control set model predictive control in power electronics," *IEEE Trans. Ind. Informat.*, vol. 9, no. 2, pp. 1003–1016, 2013.

[3] P. Cortés *et al.*, "Predictive control in power electronics and drives," *IEEE Trans. Ind. Electron.*, vol. 55, no. 12, pp. 4312–4324, 2008.

[4] T. Geyer, *Model Predictive Control of High Power Converters and Industrial Drives*. Wiley, 2016.